Forcing FP Exceptions in HeCBench's `fluidSim-cuda`

A Deliberately Unstable LBM Variant, Observed Under Nixnan

SC26 Tutorial / Nixnan Experiments

1 Executive Summary

`hec_fluidSim/` is a local copy of HeCBench's `fluidSim-cuda` benchmark (a D2Q9 Lattice-Boltzmann Method fluid solver), kept as this tutorial's own test area since `external/HeCBench` is a submodule we cannot push changes to. Run as shipped, this benchmark never produces a single FP exception — its default relaxation parameter is safely inside the scheme's stability bound. This report documents:

1. **How a genuinely exception-producing variant was built** (`main_overflow.cu`): pushing the BGK relaxation parameter `omega` far outside its $0 < \omega < 2$ stability bound turns the per-iteration collision update from a damping recurrence into an amplifying one, overflowing double's range within about 40 iterations and driving the local density through exactly zero, which the kernel's own undivided-by-zero-guarded `u.x = (...)/rho` then turns into a second, independent exception source (Section 3).
2. **The result, confirmed two ways**: a standalone host-side scan (580,644 of 589,824 output cells are NaN) and an independent unsampled nixnan run reporting the same failure at the SASS instruction level — 1,249 distinct FP64 NaN sites (16.7 million repeats) in the kernel's own `DFMA/DMUL/DADD` instructions (Section 4).
3. **Two binary-level false positives**, both real, but with different, distinguishable root causes (Section 5): one is a second confirmed instance of an already-documented `lu_solve_gpu` finding (a compiler register-move idiom misread as genuine `f32` arithmetic); the other is a new, only partially diagnosed anomaly in nixnan's own type-reporting machinery, where a definitively `f64`-classified instruction (`DFMA`, confirmed directly against nixnan's instruction table) is nonetheless *labeled* `f16` in the live error report.

2 Provenance and Algorithm

`fluidSim-cuda` is a D2Q9 Lattice-Boltzmann Method (LBM) fluid simulation on a fixed 256×256 2D grid, copied from zjin-lcf/HeCBench, path `src/fluidSim-cuda`, commit `2b73f4d` (see `hec_fluidSim/README.md` for full attribution and licensing — the benchmark itself traces to a 2009 AMD APP SDK sample). Each grid cell holds a particle-distribution function over the 9 D2Q9 discrete velocities. One CUDA kernel, `lbm` (`kernels.cu`), fuses the two classic LBM steps every iteration:

- **Collision**: for fluid (non-boundary) cells, computes local density ρ and velocity u from the current distributions, then relaxes each distribution toward its local equilibrium value (`ced()`, implementing $f_{\text{eq}} = \rho\, w\,(1 + 3(e{\cdot}u) + 4.5(e{\cdot}u)^2 - 1.5|u|^2)$) via

$$f_{\text{new}} = (1-\omega)\, f_{\text{old}} + \omega\, f_{\text{eq}}.$$

- **Streaming**: pushes each cell's post-collision values to its 8 neighbors along the corresponding D2Q9 direction, with the domain's boundary ring of cells (`h_type`) handled via bounce-back instead.

The driver, `fluidSim()`, launches `lbm` once per requested iteration, ping-ponging between two distribution buffers each step. Upstream's own `main.cu` uses $\omega = 1.2$ and a small initial velocity ($u_0 = (0.01, 0.01)$) — both safely inside the scheme's stable regime, which is exactly why the unmodified benchmark never produces an FP exception under nixnan, regardless of iteration count.

3 How the NaN-Producing Variant Was Made

3.1 The stability bound, and how to break it on purpose

The collision update $f_{\text{new}} = (1-\omega) f_{\text{old}} + \omega f_{\text{eq}}$ is a textbook damped recurrence: any deviation $\delta = f_{\text{old}} - f_{\text{eq}}$ between a cell's actual and equilibrium distributions carries forward as $(1-\omega)\,\delta$ each step. For $0 < \omega < 2$, $|1-\omega| < 1$ and deviations shrink every iteration — this is why the scheme is stable at upstream's $\omega = 1.2$. Push ω far outside that range and $|1-\omega|$ becomes a large multiplier instead: deviations grow roughly $|1-\omega|\times$ per iteration.

`main_overflow.cu` (a copy of `main.cu`, with the `#ifdef VERIFY` reference-comparison path removed — see Section 4 for why a GPU/CPU comparison stops being meaningful here) sets

$$\omega = 10^8$$

— about 8×10^7 past the stability bound. The simulation's initial state is constructed to already sit *exactly* at equilibrium (the same `computefEq()` call fills both the initial distributions and, implicitly, their own equilibrium target), so the very first collision is a no-op regardless of ω; deviations only start appearing once streaming and boundary bounce-back mix values between neighboring cells, from iteration 2 onward. Once they do, at $|1-\omega| \approx 10^8$ per iteration, an initial deviation on the order of 10^{-3} reaches double's $\sim 1.8 \times 10^{308}$ ceiling in

$$\log_{10^8}\left(\frac{10^{308}}{10^{-3}}\right) \approx \frac{311}{8} \approx 39$$

iterations. `MakeOverflow`'s `run` target uses 60 for margin.

3.2 A second exception source: division by (an oscillating) density

`kernels.cu`'s `lbm` kernel computes velocity from density with no zero-guard:

```
u.x = (dot(f1234, x1234) + dot(f5678, x5678)) / rho;
u.y = (dot(f1234, y1234) + dot(f5678, y5678)) / rho;
```

Because $(1-\omega)$ is a large *negative* number once $\omega > 1$, the collision update flips sign every iteration as it diverges — so the locally-summed density ρ (itself a sum of the nine distributions) does not just grow in magnitude, it oscillates in sign while doing so, and is expected to cross exactly 0.0 at some cell, some iteration, before overflow finishes the job. That turns `u.x`/`u.y` into a direct $x/0$ division — a second, independent route to Infinity/NaN, on top of plain multiplicative overflow.

3.3 Build

```
# MakeOverflow (new here; see README.md) -- no -DVERIFY
make -f MakeOverflow run # ./main_overflow 60
```

4 Results

4.1 Standalone sanity check (independent of nixnan)

`main_overflow.cu` scans its own output for non-finite cells before nixnan ever enters the picture:

Output cells	Count
Finite	9,180
NaN	580,644
Infinity	0
Total	589,824

98.4% of all output cells are NaN after 60 iterations at $\omega = 10^8$ — confirming the mechanism works, independent of any instrumentation. (No raw Infinity survives to the *final* output buffer; see Section 5 below for why nixnan's own live report, taken *during* the run rather than at the end, does show Infinity.) This is also why `main_overflow.cu` does not use `-DVERIFY`: once both the GPU and a CPU `reference()` implementation are diverging chaotically, floating-point non-associativity makes them diverge in different, equally meaningless ways — a GPU/CPU numerical match stops being a meaningful question once the scheme itself is unstable.

4.2 Nixnan (unsampled, default exception checking)

```
LD_PRELOAD=../external/nixnan/nixnan.so ./main_overflow 60 \
  > Log/fluidSim-overflow-60iter-nixnan-default-exceptions.log 2>&1
```

Format	NaN (repeats)	Infinity (repeats)	−Infinity (repeats)	Subnormal (repeats)
FP16	17 (402,319)	8 (1,067)	0 (0)	0 (0)
FP32	6 (268,218)	0 (0)	0 (0)	0 (0)
FP64	1,249 (16,701,143)	831 (433,209)	824 (444,362)	26 (28,314)

Distinct-site counts are how many unique (instruction, operand) pairs first triggered each exception type; repeat counts are total occurrences across the whole 60-iteration run. The FP64 row is the genuine result: 1,249 distinct sites and 16.7 million total NaN occurrences in the kernel's own `DFMA/DMUL/DADD` instructions — exactly `lbm`'s own collision/streaming arithmetic, all double-precision, as expected. The 26 distinct Subnormal sites are consistent with values genuinely passing through the full extreme- magnitude spectrum (subnormal → normal → overflow → NaN) as the amplification runs away. The FP16 and FP32 rows are false positives, covered next.

5 A Binary-Level False Positive, Revisited

`lbm` is written entirely in `double`; every genuine computation in it is FP64. The nonzero FP16/FP32 rows above are therefore both false positives in the sense already documented for `lu_solve_gpu` (`lu_solve_gpu/lu_solve_gpu_report.tex`, "An FP32 Artifact of Binary Instrumentation"): *a nonzero count in a format the source never uses does not necessarily mean that format's arithmetic ran.* But the two rows have different, and only one of them fully understood, root causes.

5.1 The FP32 case: the same register-move idiom, confirmed again

Every FP32-classified error line in this run is one of exactly two instruction texts:

```
FFMA R6, RZ, R5, R15
FFMA R6, RZ, R5, R33
```

This is precisely `lu_solve_gpu`'s documented pattern: `ptxas`'s "multiply-by-the-zero-register" idiom ($0 \times R5 + R15 = R15$), a plain register *move*, here used to shuffle one 32-bit half of a 64-bit double across the register file — not a genuine 32-bit computation. Because nixnan classifies by SASS opcode

(FFMA → f32, unconditionally, per src/instructions.h), it correctly identifies the instruction as f32-typed and correctly notices the operand's bit pattern decodes as NaN — it is simply never told that this particular FFMA is standing in for a double-register move, not real float arithmetic. This is a second, independent confirmation of the lu_solve_gpu finding, in an entirely different codebase.

5.2 The FP16 case: a distinct, only partially diagnosed anomaly

Every FP16-classified error line, by contrast, is one of exactly nine instruction texts, all of the same shape:

```
DFMA R8, R14, -1.5, R8
DFMA R10, R14, -1.5, R10
DFMA R12, R14, -1.5, R12
DFMA R14, R14, -1.5, R6
DFMA R36, R14, -1.5, R36
DFMA R38, R14, -1.5, R38
DFMA R40, R14, -1.5, R40
DFMA R42, R14, -1.5, R42
DFMA R50, R14, -1.5, R50
```

This is *not* the same phenomenon. DFMA is not an idiom here: the constant -1.5 is exactly the $-1.5|u|^2$ coefficient in ced()'s equilibrium formula (Section 3), and R14 plausibly holds $|u|^2$ — this is genuine, textbook double-precision arithmetic computing part of the equilibrium distribution, not a register-shuffle trick. Directly decoding nixnan's own instruction table (src/instructions.h, extracted from its byte-array encoding) confirms this: the DFMA entry classifies *all four* operand positions as f64, unconditionally, with no f16-typed DFMA pattern anywhere in the table:

```
"(DFMA(?:\.(RP|RM|RZ))?)": {
  "registers": [
    {"type": "f64", "count": 1}, {"type": "f64", "count": 1},
    {"type": "f64", "count": 1}, {"type": "f64", "count": 1}
  ],
  "description": "Double-precision fused multiply-add with optional FTZ and saturation
      "
}
```

So the table itself never says DFMA is f16. Reading nixnan.cu's error-reporting code (around the recv_thread_fun loop) shows the type string printed per report is *not* looked up from the table at report time; it comes from

```
type = type_to_string.at(recorder->get_type(id, ei->operand()));
```

— a runtime lookup keyed by an internal instruction id and the operand slot, populated once when the instruction was first instrumented. That the table unambiguously says f64 while this lookup sometimes returns f16 for the same instruction text points to the id→type mapping itself, not the classification table, as the likely site of the bug — e.g. an id collision between this DFMA and an unrelated, genuinely f16-typed instruction elsewhere in the same kernel, so that the wrong stored type is retrieved for this operand. **This report does not go further than that**: the detection itself is not in question (a real NaN genuinely appears at this operand, at this instruction, in this many-times-repeated pattern) — only the f16 *label* attached to it is wrong, and exactly why is not fully diagnosed here. Worth raising with nixnan's maintainers as a reproducible case: this exact 60-iteration run, this exact instruction list.

6 Conclusion

1. fluidSim-cuda's LBM scheme is stable, and exception-free under nixnan, entirely because of its default relaxation parameter choice — not because the kernel is somehow immune to FP exceptions. Pushing ω outside its $0 < \omega < 2$ stability bound reliably produces both overflow (from

the amplifying collision recurrence) and division-by-(oscillating-through-)zero (from the kernel's unguarded `u.x = (...)/rho`), confirmed both by a standalone host-side scan and by nixnan's own instruction-level report.

2. Binary-level FP exception detection can still be trusted at the level that matters most — *a real exception really occurred, at this instruction* — even when its *format label* is wrong. Two false-positive format labels were found in this one run, with two different causes: one already-documented and well-understood (a compiler register-move idiom, `lu_solve_gpu`'s finding, reproduced here independently); one new and only partially diagnosed (a likely `id`-collision in nixnan's own type-recording machinery, isolated here to a specific, reproducible instruction pattern but not fully root-caused).

Reproduction

```
cd hec_fluidSim
make -f MakeOverflow run
LD_PRELOAD=../external/nixnan/nixnan.so ./main_overflow 60 \
  > Log/fluidSim-overflow-60iter-nixnan-default-exceptions.log 2>&1
```

Requires a built `../external/nixnan/nixnan.so`. See `README.md` in this directory for provenance, licensing, and a description of every file (including `MakeShort` and `run_reduced.sh`, unrelated to this report's exception-forcing variant).